Assignment No: 5

Deployment and Scaling of a Containerized Web Application Using Kubernetes

Aim

To deploy the Dockerized Student Task Manager on Kubernetes using Minikube, configure scaling and make it accessible to users.

Output

1. Start Minikube and verify cluster

```
aditya@aditya:~/devops-task-manager$ minikube start
* minikube v1.33.1 on Ubuntu 22.04
* Using the docker driver based on existing profile
* Done! kubectl is now configured to use "minikube" cluster
aditya@aditya:~/devops-task-manager$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
aditya@aditya:~/devops-task-manager$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   2m    v1.30.0
```

2. Load image

```
aditya@aditya:~/devops-task-manager$ minikube image load task-manager:1.0
aditya@aditya:~/devops-task-manager$ minikube image ls
docker.io/library/task-manager:1.0
```

3. Deploy application

```
aditya@aditya:~/devops-task-manager$ kubectl apply -f deployment.yaml
deployment.apps/task-manager-deployment created
aditya@aditya:~/devops-task-manager$ kubectl get deployments
NAME                     READY   UP-TO-DATE    AVAILABLE  AGE
task-manager-deployment   2/2     2             2          20s
aditya@aditya:~/devops-task-manager$ kubectl get pods
NAME                                    READY    STATUS   RESTARTS   AGE
task-manager-deployment-xxxxx-aaaaa       1/1      Running  0          20s
task-manager-deployment-xxxxx-bbbbb       1/1      Running  0          20s
```

4. Create service and access

```
aditya@aditya:~/devops-task-manager$ kubectl apply -f service.yaml
service/task-manager-service created
aditya@aditya:~/devops-task-manager$ kubectl get svc
NAME                   TYPE        CLUSTER-IP      PORT(S)          AGE
kubernetes             ClusterIP   10.96.0.1       443/TCP          5m
task-manager-service   NodePort    10.101.45.12    3000:31245/TCP   10s
aditya@aditya:~/devops-task-manager$ minikube service task-manager-service --url
http://192.168.49.2:31245
Student Task Manager
Application successfully deployed on Kubernetes.
```

5. Scale to 4 replicas

```
aditya@aditya:~/devops-task-manager$ kubectl scale deployment task-manager-deployment --replicas=4
deployment.apps/task-manager-deployment scaled
aditya@aditya:~/devops-task-manager$ kubectl get deployments
NAME                     READY   UP-TO-DATE   AVAILABLE   AGE
task-manager-deployment   4/4     4            4           2m
aditya@aditya:~/devops-task-manager$ kubectl get pods
task-manager-deployment-xxxxx-aaaaa       1/1     Running   0   2m
task-manager-deployment-xxxxx-bbbbb       1/1     Running   0   2m
task-manager-deployment-xxxxx-ccccc       1/1     Running   0   15s
task-manager-deployment-xxxxx-ddddd       1/1     Running   0   15s
```

6. Self-healing

```
aditya@aditya:~/devops-task-manager$ kubectl delete pod task-manager-deployment-xxxxx-aaaaa
pod "task-manager-deployment-xxxxx-aaaaa" deleted
aditya@aditya:~/devops-task-manager$ kubectl get pods
task-manager-deployment-xxxxx-bbbbb       1/1     Running             0   3m
task-manager-deployment-xxxxx-ccccc       1/1     Running             0   1m
task-manager-deployment-xxxxx-ddddd       1/1     Running             0   1m
task-manager-deployment-xxxxx-eeeee       1/1     ContainerCreating   0   2s
Kubernetes detected the missing Pod.
New Pod created automatically.
Desired Replicas: 4
Running Pods: 4
```

7. Pod logs and stop

```
aditya@aditya:~/devops-task-manager$ kubectl logs task-manager-deployment-xxxxx-bbbbb
Student Task Manager running on port 3000
aditya@aditya:~/devops-task-manager$ minikube stop
* Stopping node "minikube" ...
* 1 node stopped.
```

Conclusion

The application was deployed on a Minikube cluster with a NodePort service, scaled from 2 to 4 replicas, and Kubernetes self-healing was demonstrated.